# AJAY.I - 71812401007_MICROCONTROLLER_AND_PROGRAMMING

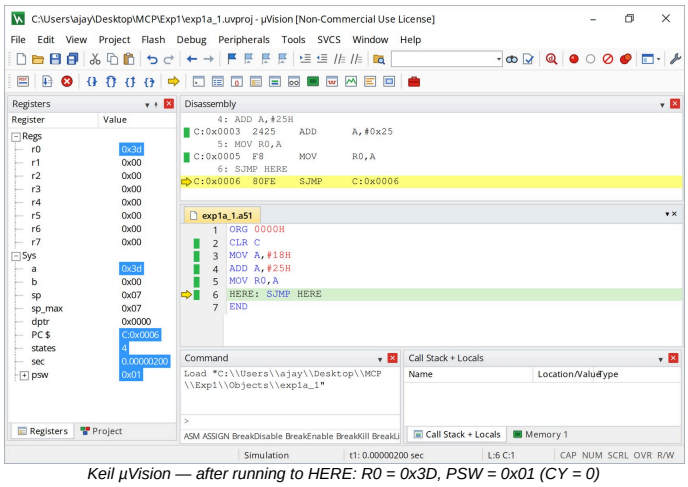
## Experiment 1a — Addition and subtraction of two 8-bit numbers

### 1. Addition without carry

```
ORG 0000H
CLR C
MOV A,#18H
ADD A,#25H
MOV R0,A
HERE: SJMP HERE
END
```

Output: `R0 = 3DH`, Carry = `0`

**OUTPUT**



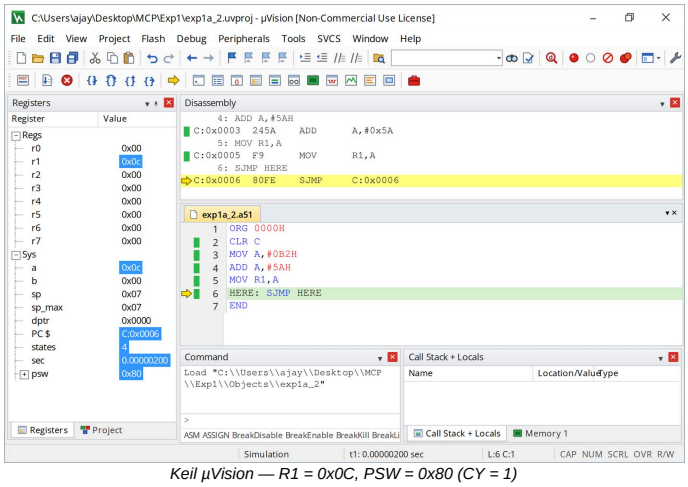

*Keil µVision — after running to HERE: R0 = 0x3D, PSW = 0x01 (CY = 0)*

### 2. Addition with carry

```
ORG 0000H
CLR C
MOV A,#0B2H
ADD A,#5AH
MOV R1,A
HERE: SJMP HERE
END
```

Output: `R1 = 0CH`, Carry = `1`

**OUTPUT**



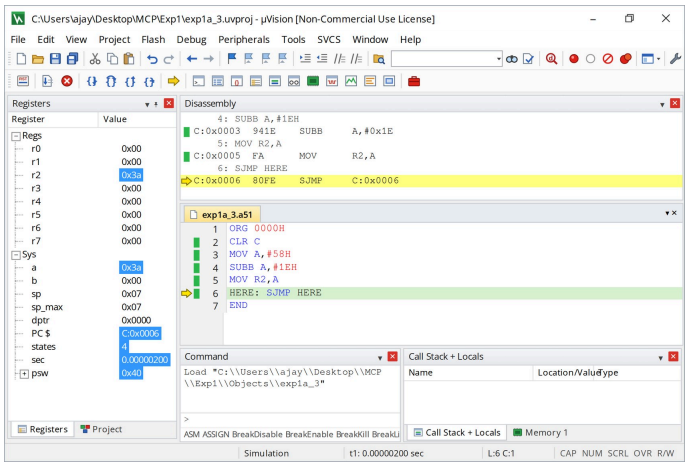

*Keil µVision — R1 = 0x0C, PSW = 0x80 (CY = 1)*

### 3. Subtraction without borrow

```
ORG 0000H
CLR C
MOV A,#58H
SUBB A,#1EH
MOV R2,A
HERE: SJMP HERE
END
```

Output: `R2 = 3AH`, Carry = `0`

**OUTPUT**




*Keil µVision — R2 = 0x3A, CY = 0*

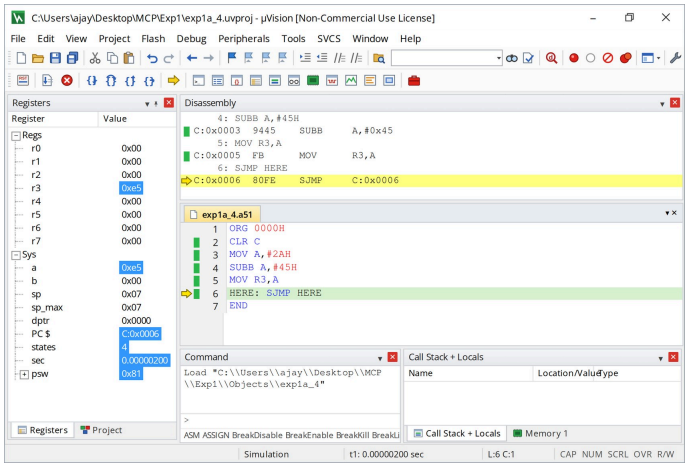
### 4. Subtraction with borrow

```
ORG 0000H
CLR C
MOV A,#2AH
SUBB A,#45H
MOV R3,A
HERE: SJMP HERE
END
```

Output: `R3 = E5H`, Carry = `1`

Remember: In 8051 subtraction, `SUBB` subtracts the operand and the carry flag. Clear carry before subtraction when you want a straightforward subtraction.

**OUTPUT**




*Keil µVision — R3 = 0xE5, PSW = 0x81 (CY = 1, borrow)*

## Experiment 1b — Multiplication and division of two 8-bit numbers

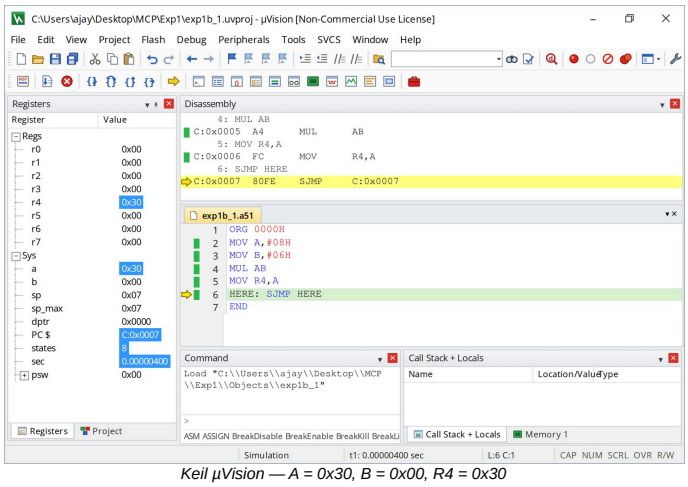
Record pages 4–6 · Keil Assembly · Use each program separately.

### 1. Multiplication without overflow

```
ORG 0000H
MOV A,#08H
MOV B,#06H
MUL AB
MOV R4,A
HERE: SJMP HERE
END
```

Output: `A = 30H`, `B = 00H`, `R4 = 30H`

**OUTPUT**



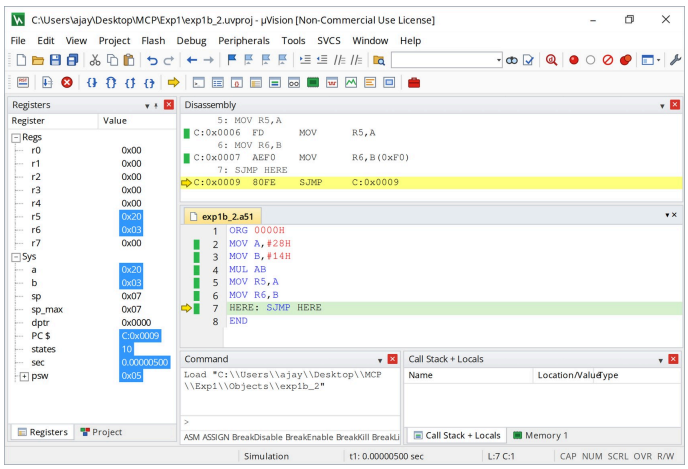

*Keil µVision — A = 0x30, B = 0x00, R4 = 0x30*

## 2. Multiplication with overflow

```
ORG 0000H
MOV A,#28H
MOV B,#14H
MUL AB
MOV R5,A
MOV R6,B
HERE: SJMP HERE
END
```

Output: `A = 20H`, `B = 03H`, `R5 = 20H`, `R6 = 03H`

**OUTPUT**



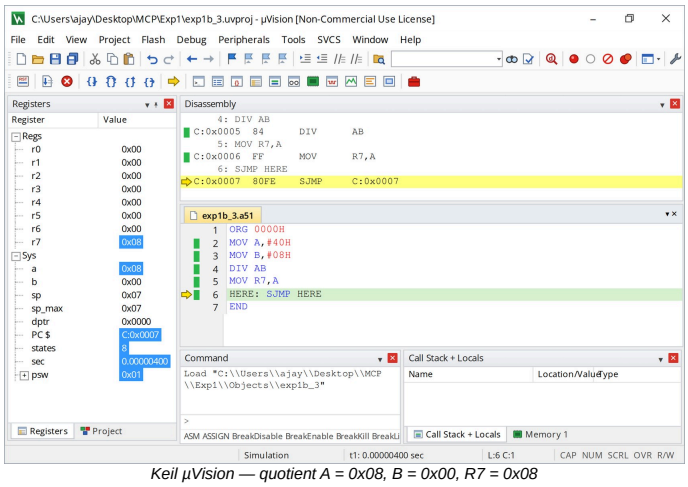

*Keil µVision — A = 0x20, B = 0x03, R5 = 0x20, R6 = 0x03 (OV = 1)*

## 3. Division without remainder

```
ORG 0000H
MOV A,#40H
MOV B,#08H
DIV AB
MOV R7,A
HERE: SJMP HERE
END
```

Output: Quotient `A = 08H`, remainder `B = 00H`, `R7 = 08H`

**OUTPUT**




*Keil µVision — quotient A = 0x08, B = 0x00, R7 = 0x08*

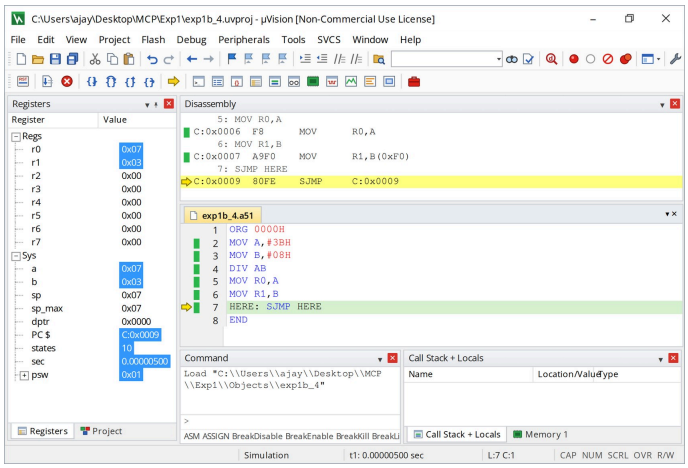
## 4. Division with remainder

```
ORG 0000H
MOV A,#3BH
MOV B,#08H
DIV AB
MOV R0,A
MOV R1,B
HERE: SJMP HERE
END
```

Output: Quotient `R0 = 07H`, remainder `R1 = 03H`

Remember: `MUL AB` stores the 16-bit product across `B:A`. `DIV AB` stores the quotient in `A` and remainder in `B`.

**OUTPUT**




*Keil µVision — R0 = 0x07 (quotient), R1 = 0x03 (remainder)*

# Experiment 1c — Logical AND and OR

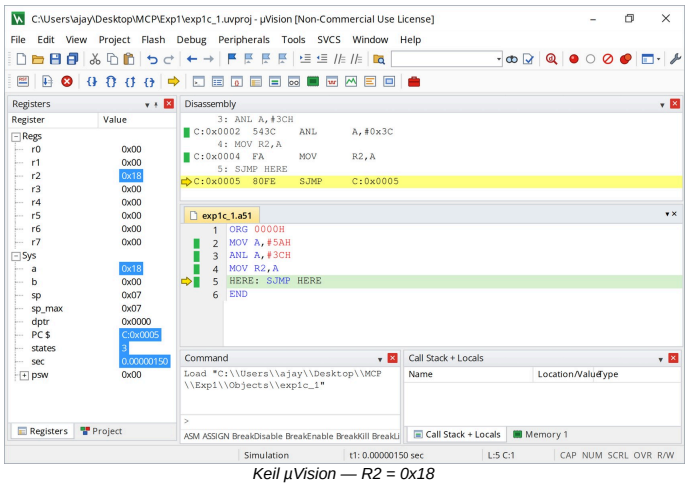
Record pages 7–8 · Keil Assembly.

## 1. Logical AND

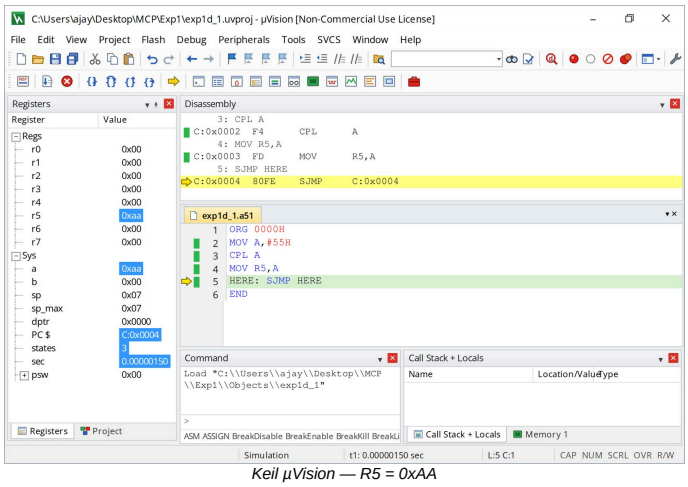
```
ORG 0000H
MOV A,#5AH
ANL A,#3CH
MOV R2,A
HERE: SJMP HERE
END
```

Output: `R2 = 18H`

**OUTPUT**




**OUTPUT**

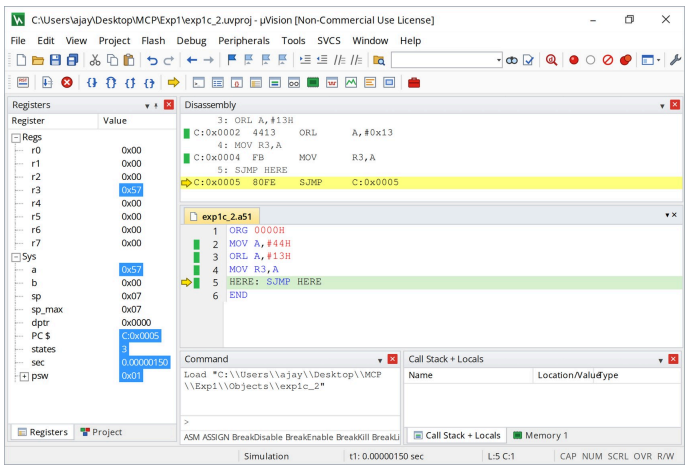



## 2. Logical OR

```
ORG 0000H
MOV A,#44H
ORL A,#13H
MOV R3,A
HERE: SJMP HERE
END
```

Output: `R3 = 57H`

**OUTPUT**

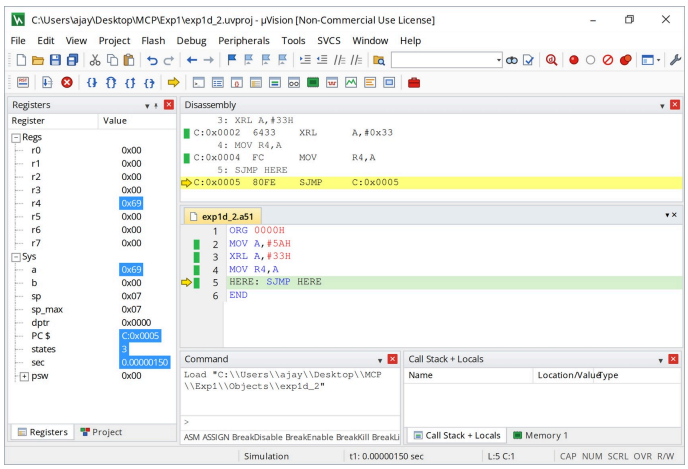



## 2. Logical XOR

```
ORG 0000H
MOV A,#5AH
XRL A,#33H
MOV R4,A
HERE: SJMP HERE
END
```

Output: `R4 = 69H`

**OUTPUT**




# Experiment 1d — Logical NOT and XOR

Record pages 9–10 · Keil Assembly.

## 1. Logical NOT (complement)

```
ORG 0000H
MOV A,#55H
CPL A
MOV R5,A
HERE: SJMP HERE
END
```

Output: `R5 = AAH`

# Experiment 2 — Data transfer between memory blocks

Record pages 11–12 · Keil Assembly.

## 2a. Transfer four bytes from 60H to 70H

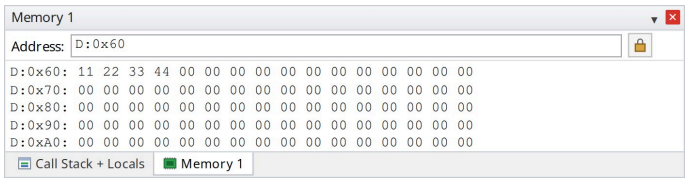
```
ORG 0000H
MOV R0,#60H
MOV R1,#70H
MOV R2,#04H
L1: MOV A,@R0
MOV @R1,A
INC R0
INC R1
DJNZ R2,L1
HERE: SJMP HERE
END
```

Sample memory setup: Before running, place `11H, 22H, 33H, 44H` in memory addresses `60H–63H`.

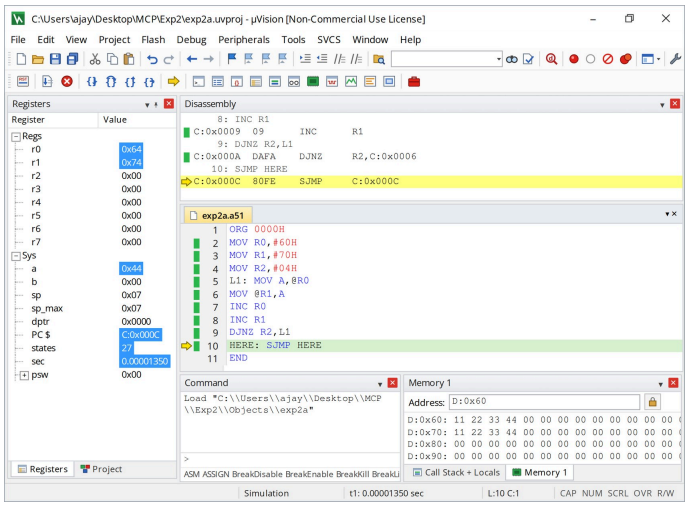
Expected output:

| Source | Destination |
|---|---|
| `60H = 11H` | `70H = 11H` |
| `61H = 22H` | `71H = 22H` |
| `62H = 33H` | `72H = 33H` |
| `63H = 44H` | `73H = 44H` |

**OUTPUT**




*Memory window — before execution (60H–63H hold 11 22 33 44, 70H–73H empty)*




*Keil µVision — after execution (70H–73H now hold 11 22 33 44)*

## 2b. Exchange four bytes between 50H and 80H

```
ORG 0000H
MOV R0,#50H
MOV R1,#80H
MOV R2,#04H
L1: MOV A,@R0
MOV B,A
MOV A,@R1
MOV @R0,A
MOV A,B
MOV @R1,A
INC R0
INC R1
DJNZ R2,L1
HERE: SJMP HERE
END
```

Sample memory setup:

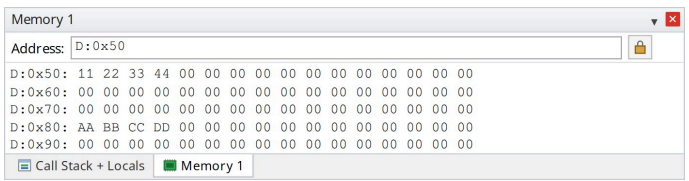
- `50H–53H`: `11H, 22H, 33H, 44H`
- `80H–83H`: `AAH, BBH, CCH, DDH`

Expected output: The two blocks exchange their contents.

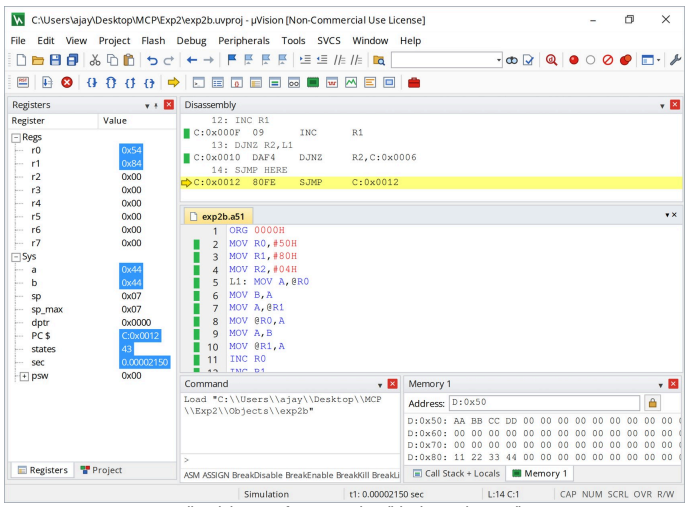
| Address range | Before | After |
|---|---|---|
| `50H–53H` | `11 22 33 44` | `AA BB CC DD` |
| `80H–83H` | `AA BB CC DD` | `11 22 33 44` |

Keil tip: Open the Memory window and inspect the internal data memory addresses above. Set the initial bytes before running.

**OUTPUT**




*Memory window — before execution*




*Keil µVision — after execution (blocks exchanged)*

## Experiment 3 — Find the largest and smallest numbers in an array

Record pages 13–15 · Keil Assembly.

For both programs, store the array length at `50H` and the array values from `60H` onward. The result is stored at `70H`.

### 3a. Find the largest number

```
ORG 0000H
MOV R2,50H
DEC R2
MOV R0,#60H
MOV A,@R0
INC R0
L1: CJNE A,@R0,L2
L2: JNC L3
MOV A,@R0
L3: INC R0
DJNZ R2,L1
MOV 70H,A
HERE: SJMP HERE
END
```

### 3b. Find the smallest number

```
ORG 0000H
MOV R2,50H
DEC R2
MOV R0,#60H
MOV A,@R0
INC R0
L1: CJNE A,@R0,L2
L2: JC L3
MOV A,@R0
L3: INC R0
DJNZ R2,L1
MOV 70H,A
HERE: SJMP HERE
END
```

Sample input for both programs:

| Memory address | Value |
|---|---|
| `50H` — array length | `04H` |
| `60H` | `12H` |
| `61H` | `45H` |
| `62H` | `08H` |
| `63H` | `32H` |

Expected results:

- Largest: `70H = 45H`
- Smallest: `70H = 08H`

These are unsigned 8-bit comparisons. The array must contain at least one element.

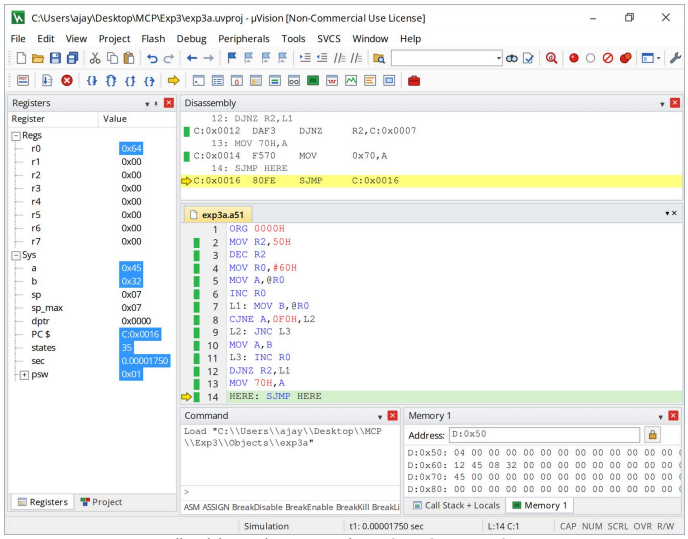
Important correction for Experiment 3: The 8051 does not support `CJNE A,@R0,L2`. Use the valid form below instead. These are the corrected, Keil-compatible programs.

### 3a. Largest number — corrected code

```
ORG 0000H
MOV R2,50H
DEC R2
MOV R0,#60H
MOV A,@R0
INC R0
```

```
L1: MOV B,@R0
CJNE A,0F0H,L2
L2: JNC L3
MOV A,B
L3: INC R0
DJNZ R2,L1
MOV 70H,A
HERE: SJMP HERE
END
```

**OUTPUT**

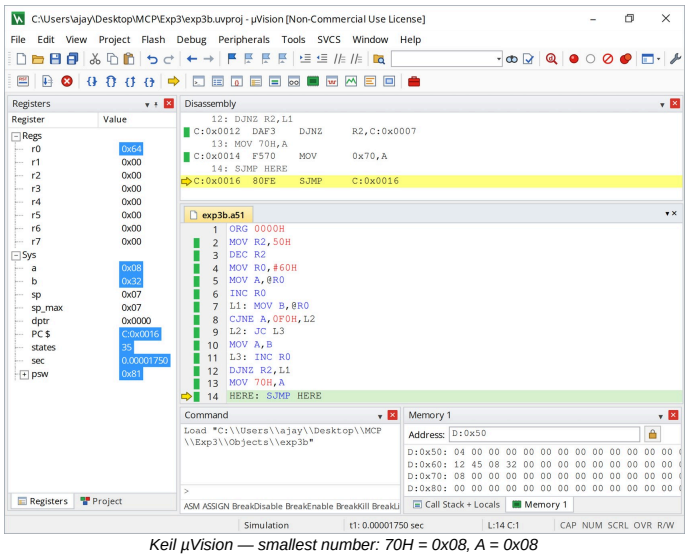



### 3b. Smallest number — corrected code

```
ORG 0000H
MOV R2,50H
DEC R2
MOV R0,#60H
MOV A,@R0
INC R0
L1: MOV B,@R0
CJNE A,0F0H,L2
L2: JC L3
MOV A,B
L3: INC R0
DJNZ R2,L1
MOV 70H,A
HERE: SJMP HERE
END
```

Here, `B` is compared through its direct address `F0H`, which is valid for `CJNE`. The carry flag tells the program which value to keep.

**OUTPUT**




## Experiment 4 — 8051 C programming for timer and UART

Record pages 16–18 · Keil C51 compiler · Save each as a `.c` file.

### 4a. Timer Mode 1 with delay

```
#include <REG51.H>

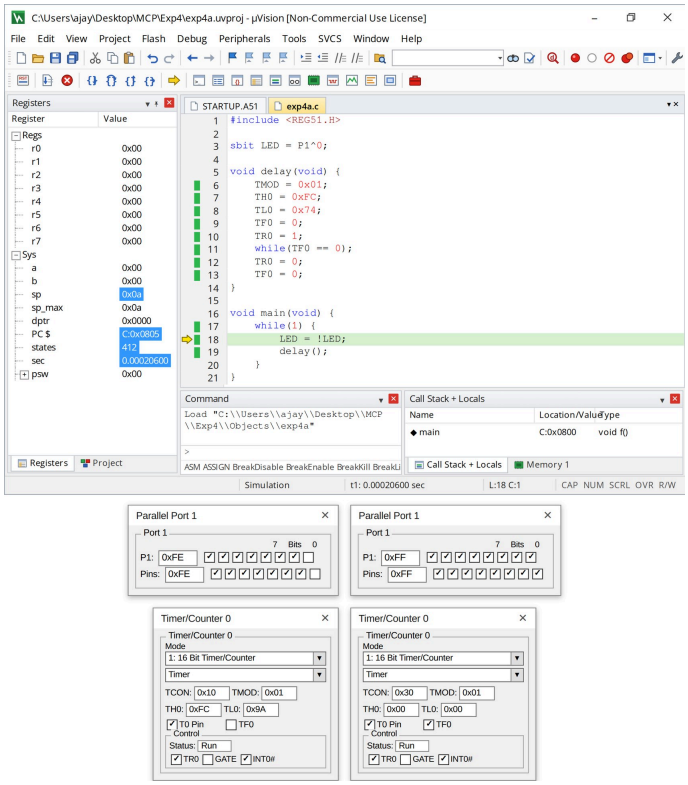
sbit LED = P1^0;

void delay(void) {
    TMOD = 0x01;
    TH0 = 0xFC;
    TL0 = 0x74;
    TF0 = 0;
    TR0 = 1;
    while(TF0 == 0);
    TR0 = 0;
    TF0 = 0;
}

void main(void) {
    while(1) {
        LED = !LED;
        delay();
    }
}
```

Expected output: `P1.0` continuously toggles between `0` and `1`. In hardware, an LED connected correctly to that pin blinks. The delay is approximately 1 ms with a 12 MHz clock, so the blink may be too fast to see clearly.

**OUTPUT**




### 4b. Timer interrupt to toggle an output pin

```
#include <REG51.H>

sbit LED = P1^0;

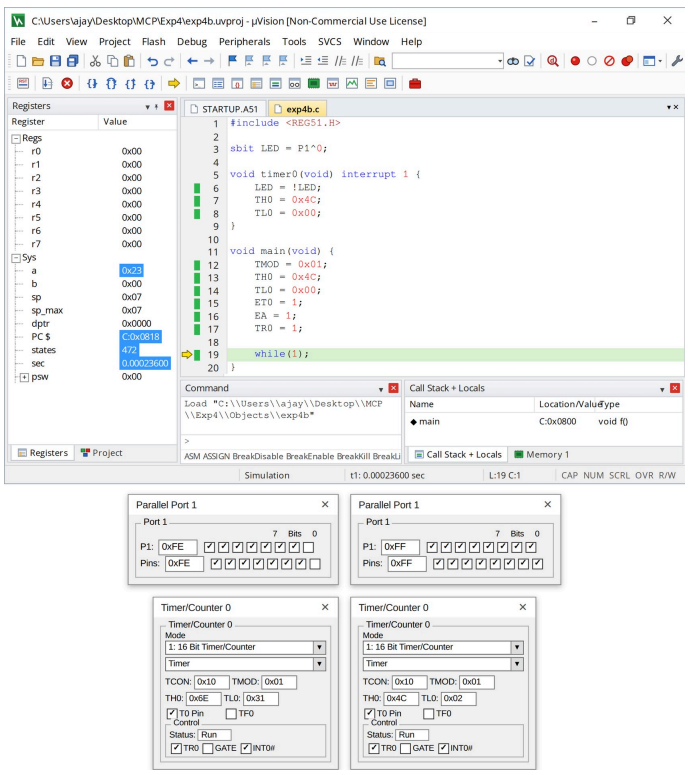
void timer0(void) interrupt 1 {
    LED = !LED;
    TH0 = 0x4C;
    TL0 = 0x00;
}

void main(void) {
    TMOD = 0x01;
    TH0 = 0x4C;
    TL0 = 0x00;
    ET0 = 1;
    EA = 1;
    TR0 = 1;

    while(1);
}
```

Expected output: The timer interrupt repeatedly toggles `P1.0`. With a 12 MHz clock, the overflow interval is approximately 50 ms, so the pin changes state about every 50 ms.

**OUTPUT**




*Keil µVision — P1.0 toggled by the Timer 0 interrupt; TH0/TL0 reloaded to 0x4C00*

### 4c. UART transmission

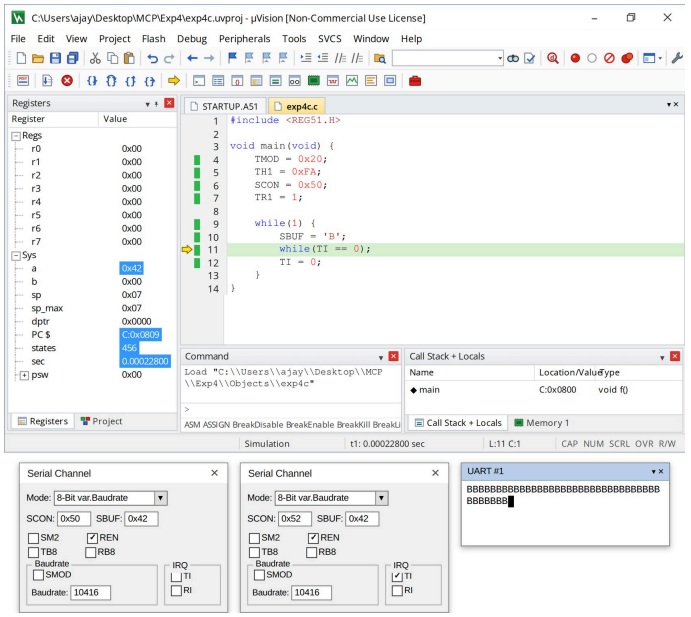
```
#include <REG51.H>

void main(void) {
    TMOD = 0x20;
    TH1 = 0xFA;
    SCON = 0x50;
    TR1 = 1;

    while(1) {
        SBUF = 'B';
        while(TI == 0);
        TI = 0;
    }
}
```

Expected output: The serial output repeatedly transmits the character `B`. In Keil, open the serial/serial-window simulation and configure it to observe the 8051 UART output. The baud rate depends on the oscillator and serial configuration.

**OUTPUT**




*Keil µVision — Serial Channel: SBUF = 0x42 (B), TI set/cleared; UART #1 window shows B…*

## Experiment 5 — 8051 C programming for port handling

Record pages 19–23 · Keil C51 compiler · Save each as a `.c` file.

### 5a. Display an ASCII number in a port

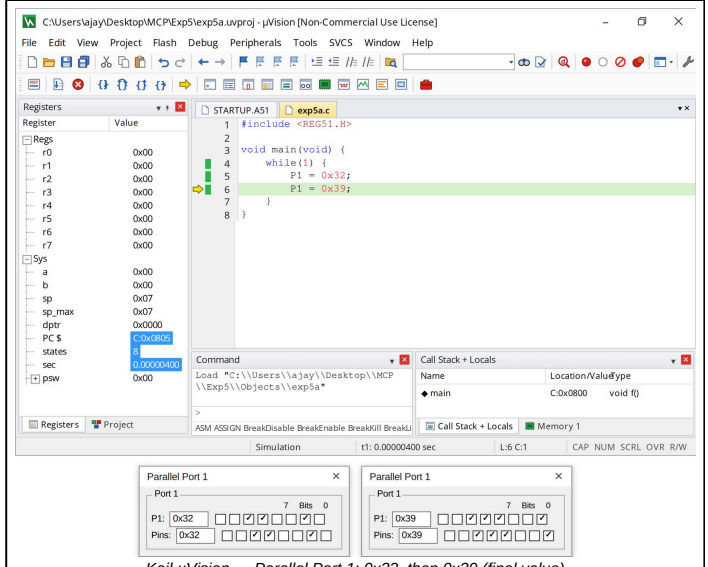
The value `29H` represents two hexadecimal digits, `2` and `9`. Their ASCII codes are `32H` and `39H`.

```
#include <REG51.H>

void main(void) {
    while(1) {
        P1 = 0x32;
        P1 = 0x39;
    }
}
```

Expected output: The port assignments are `32H` and then `39H`. Since the second assignment immediately overwrites the first, the final `P1` value will be `39H`. To observe both values clearly in simulation, single-step the code.

**OUTPUT**




*Keil µVision — Parallel Port 1: 0x32, then 0x39 (final value)*

### 5b. I/O port programming with delay

```
#include <REG51.H>

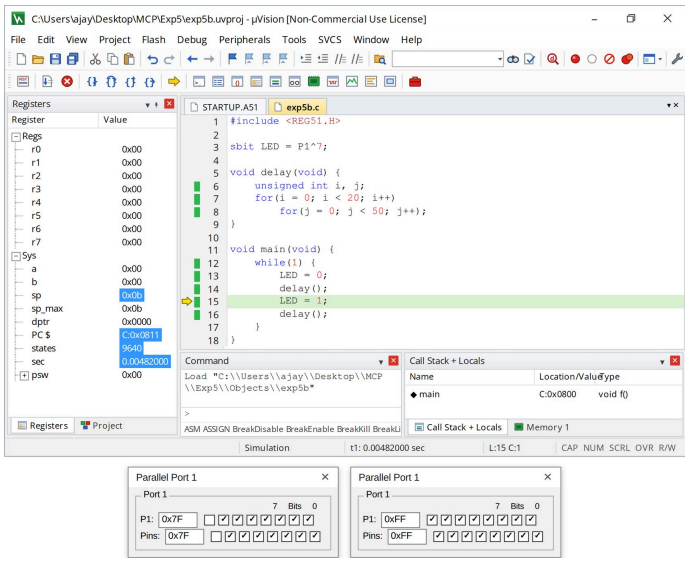
sbit LED = P1^7;

void delay(void) {
    unsigned int i, j;
    for(i = 0; i < 20; i++)
        for(j = 0; j < 50; j++);
}

void main(void) {
    while(1) {
        LED = 0;
        delay();
        LED = 1;
        delay();
    }
}
```

Expected output: `P1.7` alternates between `0` and `1`, with a software delay between changes.

**OUTPUT**




*Keil µVision — Parallel Port 1: P1.7 = 0 (0x7F) then 1 (0xFF)*

## 5c. Toggle between multiple ports

```c
#include <REG51.H>

void delay(void) {
    unsigned int i;
    for(i = 0; i < 500; i++);
}

void main(void) {
    P0 = P1 = P2 = P3 = 0x33;

    while(1) {
        P0 = ~P0;
        P1 = ~P1;
        P2 = ~P2;
        P3 = ~P3;
        delay();
    }
}
```

Expected output: All four ports alternate between `33H` (`00110011`) and `CCH` (`11001100`).

**How to run these programs in Keil µVision**

For Experiments 1–3 (Assembly):

1. Open Keil µVision and create a new project.
2. Select the 8051-compatible device used by your lab, such as Atmel AT89C51.
3. Create an Assembly source file (`.asm`) and paste in one program.
4. Add it to the project and build the target.
5. Start the debugger, run the program, and inspect registers or internal data memory.

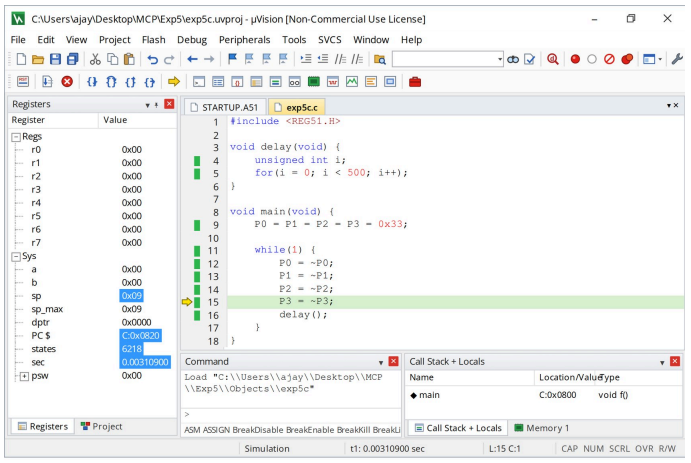
For Experiments 4–5 (C):

1. Create a C source file (`.c`) and paste in one program.
2. Make sure the project uses the Keil C51 compiler and the correct 8051 device.
3. Build the target, enter the debugger, and run.
4. Observe the port registers or serial output using the relevant simulation window.

Two important notes:

- These are separate programs: use one source file per experiment subpart. Do not paste all programs into the same source file.
- The sample outputs are expected values, not screenshots from a live Keil run. For Experiment 2 and 3, you must initialize the indicated internal memory locations before running; otherwise, the results depend on the memory's initial contents.

**OUTPUT**




*Keil µVision — Ports P0 and P1 alternate between 0x33 and 0xCC*

## 8051 Microcontroller Lab — Experiments 6a to 7b

From your photo, the kit appears to contain an 8051 trainer board, an LCD module, a USB cable, and a separate red USB interface/programmer board. That is a good starting point for your lab practical. 👍

The key is to understand that writing and compiling code in Keil is separate from transferring the program to the physical board.

### 1. Set up a fresh PC with Keil

Step 1 — Install Keil

Install Keil µVision with the C51 compiler. Open µVision and create a new project using Project → New µVision Project.

Step 2 — Select the microcontroller

Select the exact 8051-compatible chip fitted to your trainer board. Do not automatically choose AT89C52 without checking the chip marking.

Step 3 — Create and build your C program

Create a `.c` source file, add it to the project, then open Options for Target → Output and enable Create HEX File. Press F7 to build.

Step 4 — Transfer the HEX file

Connect the kit using the interface intended for your board. Install its driver or programming utility if required, select the correct chip, load the generated `.hex` file, and program the board.

Important: I can identify the general kit arrangement from your photograph, but I cannot reliably read the microcontroller's part number or the red interface board's model. The exact programming method and pin connections depend on those details. Keil alone does not necessarily program the chip through an ordinary USB cable.

### 2. Experiment 6a — Blink LEDs with delay

This follows the pin assignments in your lab record: LED1 = P1.5, LED2 = P1.4, and LED3 = P1.3.

Keil C51 · Target: AT89C52 · Save as `exp6a.c`

```c
#include <REGX52.H>

#define LED1 P1_5
#define LED2 P1_4
#define LED3 P1_3

void delay(unsigned int count)
{
    unsigned int x, y;
    for(x=0; x<count; x++)
        for(y=0; y<1275; y++);
}

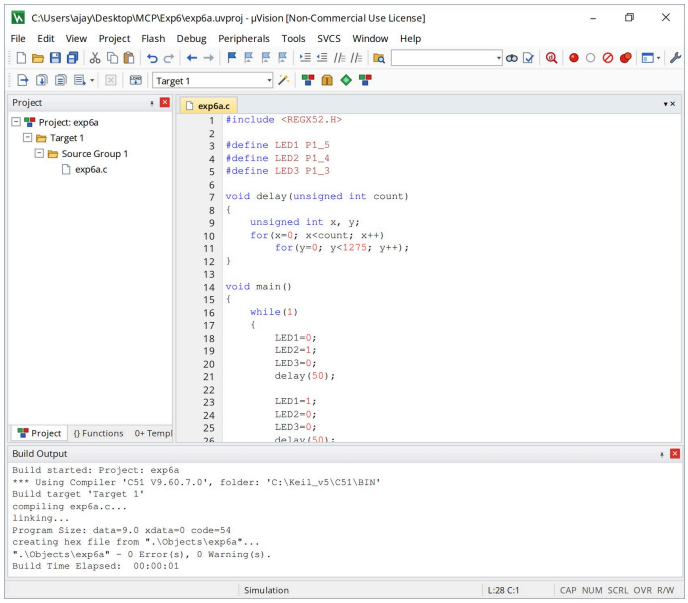
void main()
{
    while(1)
    {
        LED1=0;
        LED2=1;
```

```
        LED3=0;
        delay(50);

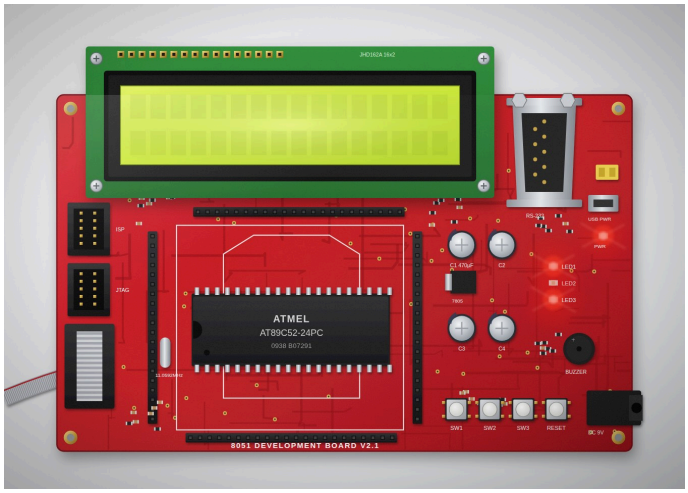
        LED1=1;
        LED2=0;
        LED3=0;
        delay(50);
    }
}
```

Expected output: LED1 and LED2 alternate; LED3 remains OFF. The code uses active-low LEDs, matching the convention in your record.

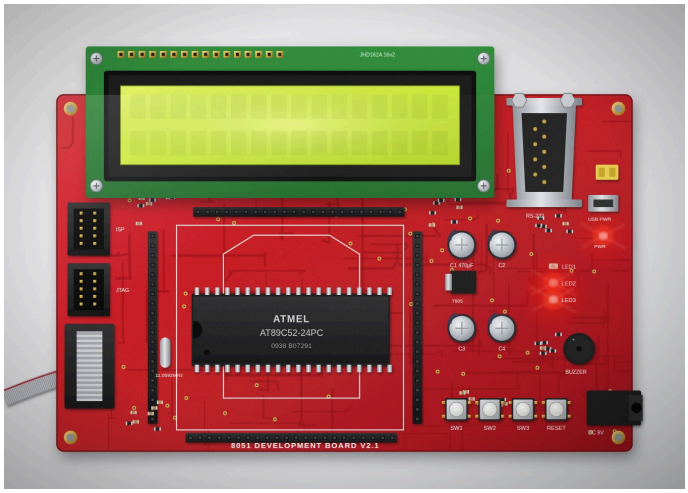
**OUTPUT**




*Keil µVision — Build Output for exp6a.c (0 errors, 0 warnings)*




*Board — state 1: LED1 and LED3 ON, LED2 OFF (active-low pins)*




*Board — state 2: LED2 and LED3 ON, LED1 OFF*

**Procedure**

1. Open Keil µVision → Project → New µVision Project.
2. Select Atmel → AT89C52.
3. Create `exp6a.c` and paste the code.
4. Add the source file to the project.
5. In Options for Target → Output, tick Create HEX File.
6. Press F7. Confirm there are no errors.
7. Use the kit's supported programming utility to transfer the `.hex` file.
8. Reset the board and observe the LEDs.

## 3. Experiment 6b — LED control using switches

This version follows your record's switch and LED mapping: switches on P2.0–P2.2, LEDs on P1.5–P1.3, and buzzer on P1.6.

Save as `exp6b.c`

```
#include <REGX52.H>

#define SW1 P2_0
#define SW2 P2_1
#define SW3 P2_2

#define BEEP P1_6
#define LED1 P1_5
#define LED2 P1_4
#define LED3 P1_3

void delay(unsigned int count)
{
    unsigned int x, y;
    for(x=0; x<count; x++)
        for(y=0; y<1275; y++);
}

void main()
{
    SW1=1;
    SW2=1;
    SW3=1;
    BEEP=1;

    LED1=1;
    LED2=1;
    LED3=1;

    while(1)
    {
        if(SW1==0)
        {
            LED1=0; LED2=1; LED3=1;
            BEEP=0; delay(25); BEEP=1;
            while(SW1==0);
        }

        if(SW2==0)
        {
            LED1=1; LED2=0; LED3=1;
            BEEP=0; delay(25); BEEP=1;
            while(SW2==0);
        }

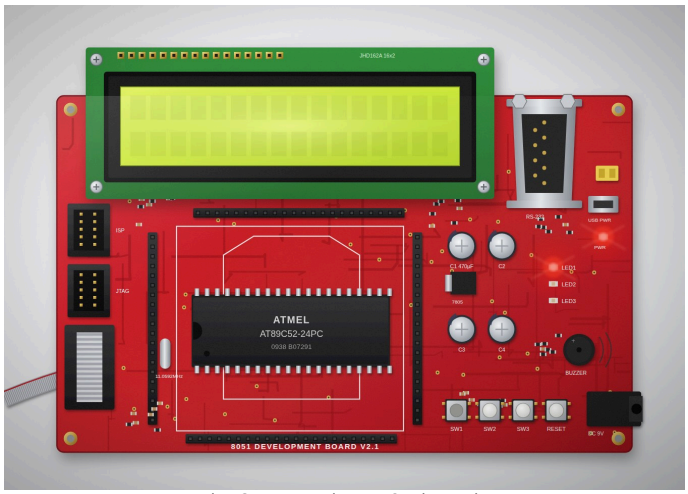
        if(SW3==0)
        {
            LED1=1; LED2=1; LED3=0;
            BEEP=0; delay(25); BEEP=1;
            while(SW3==0);
        }
    }
}
```

Expected output

- Press SW1 → LED1 turns ON and the buzzer gives a short beep.
- Press SW2 → LED2 turns ON and the buzzer gives a short beep.
- Press SW3 → LED3 turns ON and the buzzer gives a short beep.

The switch inputs are assumed to be active-low. The LEDs and buzzer must actually be connected to the pins above for this code to control them.

**OUTPUT**




*Board — SW1 pressed: LED1 ON, buzzer beeps*



*Close-up: SW1 → LED1 ON + beep*



*Close-up: SW2 → LED2 ON + beep*



*Close-up: SW3 → LED3 ON + beep*

## 4. Experiment 7a — Display text on LCD

Your record uses the LCD data port `P0`, with `EN = P1.0`, `RW = P1.1`, and `RS = P1.2`.

Save as `exp7a.c` · Target: AT89C52

```
#include <REGX52.H>

#define LCD_PORT P0
#define EN P1_0
#define RS P1_2
#define RW P1_1

void delay(unsigned int);
void lcd_command(char);
void lcd_data(char);
void lcd_display(unsigned char, char *);
void lcd_initial(void);

void main()
{
    lcd_initial();

    lcd_display(0x80, "AJAY.I");
    lcd_display(0xC0, "3rd Year");

    while(1);
}

void lcd_initial(void)
{
    delay(20);
    lcd_command(0x38);
    lcd_command(0x01);
    lcd_command(0x06);
    lcd_command(0x0C);
}

void lcd_command(char value)
{
    LCD_PORT = value;
    RS = 0;
    RW = 0;
    EN = 1;
    delay(2);
    EN = 0;
    delay(2);
}

void lcd_data(char value)
{
    LCD_PORT = value;
    RS = 1;
    RW = 0;
    EN = 1;
    delay(2);
    EN = 0;
    delay(2);
}

void lcd_display(unsigned char locn, char *dat)
{
    lcd_command(locn);

    while(*dat)
    {
        lcd_data(*dat);
        dat++;
    }
}

void delay(unsigned int count)
{
    unsigned int x, y;

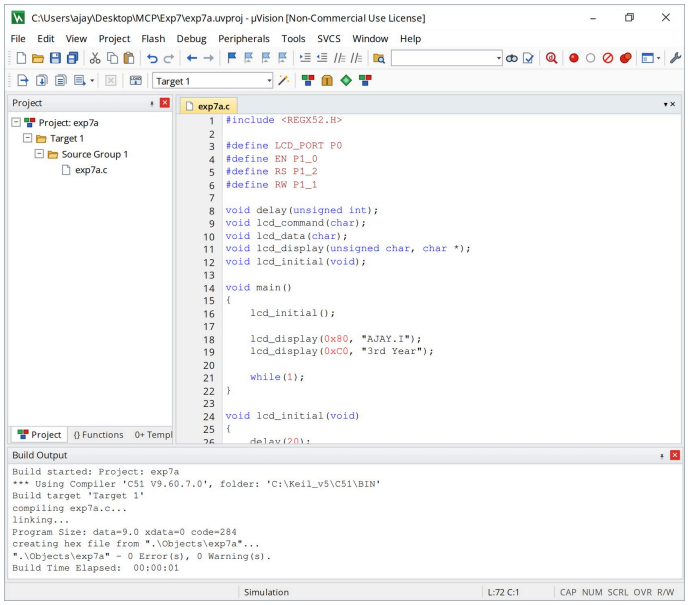
    for(x=0; x<count; x++)
        for(y=0; y<1275; y++);
}
```

Expected LCD output

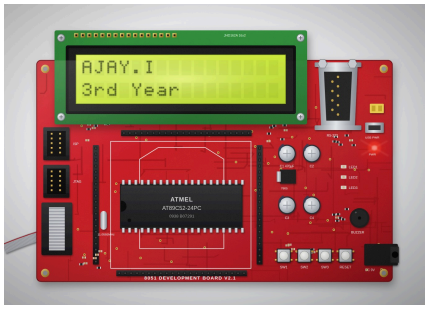
**AJAY.I**

**3rd Year**

**OUTPUT**




*Keil µVision — Build Output for exp7a.c*




*16×2 LCD shows AJAY.I / 3rd Year*

**Procedure**

1. Create a new Keil project and select AT89C52.
2. Create `exp7a.c`, paste the code and add it to the project.

3. Enable Create HEX File under Options for Target → Output.
4. Press F7 and check the build output.
5. Program the board using its supported programmer.
6. Reset the board and check the LCD.

## 5. Experiment 7b — LCD with switch control

This displays a different message when you press SW1, SW2 or SW3, as specified in your record.

Save as `exp7b.c` · Target: AT89C52

```
#include <REGX52.H>

#define LCD_PORT P0
#define EN P1_0
#define RS P1_2
#define RW P1_1

#define SW1 P2_0
#define SW2 P2_1
#define SW3 P2_2
#define BEEP P1_6

void delay(unsigned int);
void lcd_command(char);
void lcd_data(char);
void lcd_display(unsigned char, char *);
void lcd_initial(void);

void main()
{
    P2 = 0x07;
    BEEP = 1;

    lcd_initial();
    lcd_display(0x80, "Press Any Switch");

    while(1)
    {
        if(SW1 == 0)
        {
            while(SW1 == 0);
            BEEP = 0;
            delay(25);
            BEEP = 1;
            lcd_command(0x01);
            lcd_display(0x80, "Pressed Switch 1");
        }

        if(SW2 == 0)
        {
            while(SW2 == 0);
            BEEP = 0;
            delay(25);
            BEEP = 1;
            lcd_command(0x01);
            lcd_display(0x80, "Pressed Switch 2");
        }

        if(SW3 == 0)
        {
            while(SW3 == 0);
            BEEP = 0;
            delay(25);
            BEEP = 1;
            lcd_command(0x01);
            lcd_display(0x80, "Pressed Switch 3");
        }
    }
}

void lcd_initial(void)
{
    delay(20);
    lcd_command(0x38);
    lcd_command(0x01);
    lcd_command(0x06);
    lcd_command(0x0C);
}

void lcd_command(char value)
{
    LCD_PORT = value;
    RS = 0;
    RW = 0;
    EN = 1;
    delay(2);
    EN = 0;
    delay(2);
}

void lcd_data(char value)
{
    LCD_PORT = value;
    RS = 1;
    RW = 0;
    EN = 1;
    delay(2);
    EN = 0;
    delay(2);
}

void lcd_display(unsigned char locn, char *dat)
{
    lcd_command(locn);

    while(*dat)
    {
        lcd_data(*dat);
        dat++;
    }
}

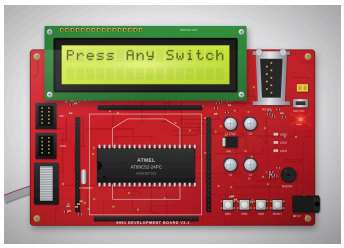
void delay(unsigned int count)
{
    unsigned int x, y;

    for(x=0; x<count; x++)
        for(y=0; y<1275; y++);
}
```

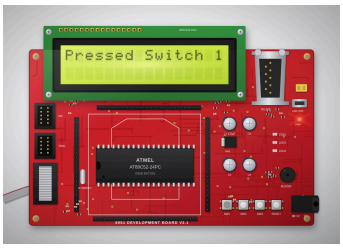
Expected output

**Pressed Switch 2**

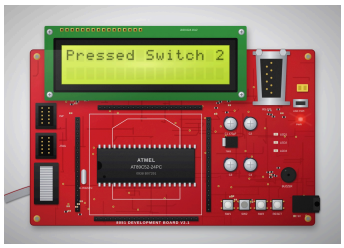
**OUTPUT**



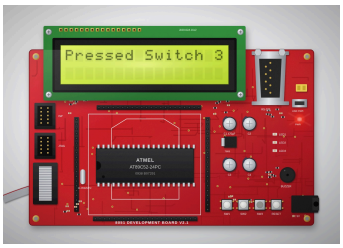

*After reset: Press Any Switch*




*SW1 pressed: Pressed Switch 1*




*SW2 pressed: Pressed Switch 2*




*SW3 pressed: Pressed Switch 3*

### Procedure

1. Create a new project in Keil and select AT89C52.
2. Paste the code into `exp7b.c`.
3. Enable Create HEX File and press F7.
4. Transfer the HEX file to the board.
5. Reset the board. The LCD should initially show `Press Any Switch`.
6. Press each switch and check the corresponding message and beep.

## 6. How to connect your physical kit

Your lab record defines the following pin assignments. Actual wiring must match the board's labels and schematic.

| Component | 8051 pin |
|---|---|
| LED1 | P1.5 |
| LED2 | P1.4 |
| LED3 | P1.3 |
| Switches SW1–SW3 | P2.0, P2.1, P2.2 |
| Buzzer | P1.6 |
| LCD data lines D0–D7 | P0 |
| LCD EN | P1.0 |
| LCD RW | P1.1 |
| LCD RS | P1.2 |

For the LCD experiments, use the board's designated LCD connector/ribbon cable if the LCD is not already wired. Do not connect wires solely from this table if your trainer board has different labels or onboard routing.

For the fresh-PC setup, remember this sequence:

1. Install Keil with the C51 compiler.
2. Create the project and select the correct chip.
3. Build the program and generate the `.hex` file.
4. Identify the board's programmer/interface and install its required driver and programming utility.
5. Select the correct COM port/chip in that utility, load the HEX file, and program the board.
6. Reset the board and verify the output.

The trainer-board manual for some similar 8051 kits uses a separate programming utility and a board-specific USB/serial interface; the precise utility depends on the chip and kit model.

## Simplest Keil codes — Experiments 8, 9 and 10

⚠️ Important: According to your lab record, Experiments 8–10 use the ARM7/LPC21xx microcontroller, not the 8051 AT89C52. So select the ARM target in Keil and use the ARM header file `LPC21xx.h`. These are separate from your 8051 experiments 6a–7b.

## Experiment 8 — ARM7 LED and LCD interfacing

Your record's Experiment 8 is ARM7 programming to interface LED and LCD.

Here is the simplest LED portion, based on the LPC21xx pin setup used in your record's ARM7 experiments.

Save as `exp8.c`

```c
#include <lpc21xx.h>

int main()
{
    IO0DIR = 0x00000080;

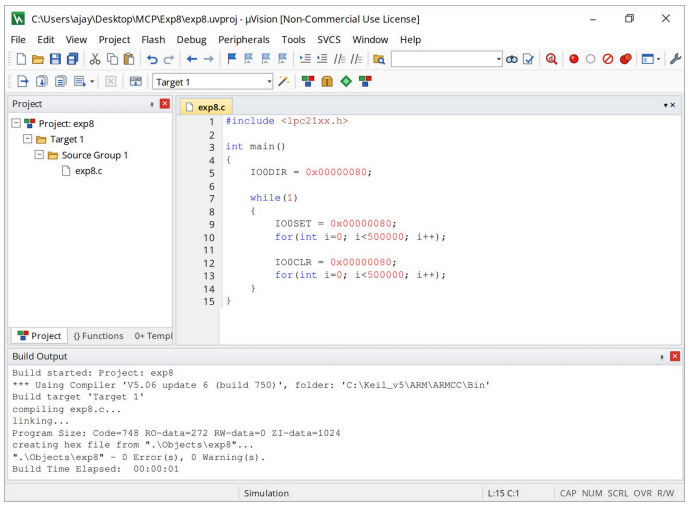
    while(1)
    {
        IO0SET = 0x00000080;
        for(int i=0; i<500000; i++);

        IO0CLR = 0x00000080;
        for(int i=0; i<500000; i++);
    }
}
```

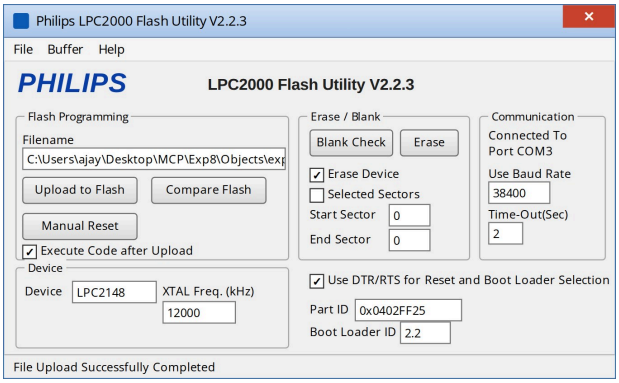
Expected output: The LED connected to P0.7 blinks ON and OFF.

Note: this is the LED portion only, not the complete LCD interfacing program.

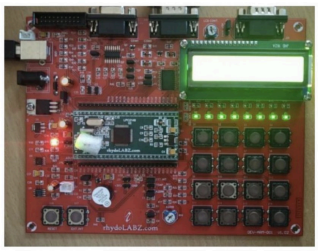
### OUTPUT



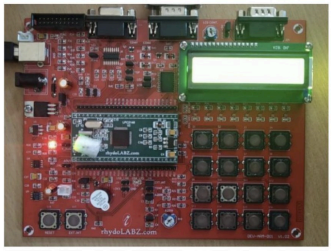

*Keil µVision — Build Output for exp8.c (ARM)*




*Philips LPC2000 Flash Utility — exp8.hex uploaded*



*LED on P0.7 ON*



*LED on P0.7 OFF*

## Experiment 9 — ARM7 buzzer interfacing

Your record uses an external interrupt to toggle the output pins. For the simplest version, this program toggles the buzzer output continuously instead.

Save as `exp9.c`

```c
#include <lpc21xx.h>

void delay(unsigned int n)
{
    unsigned int i;
    for(i=0; i<n; i++);
}

int main(void)
{
    IO0DIR |= 0x00010000;

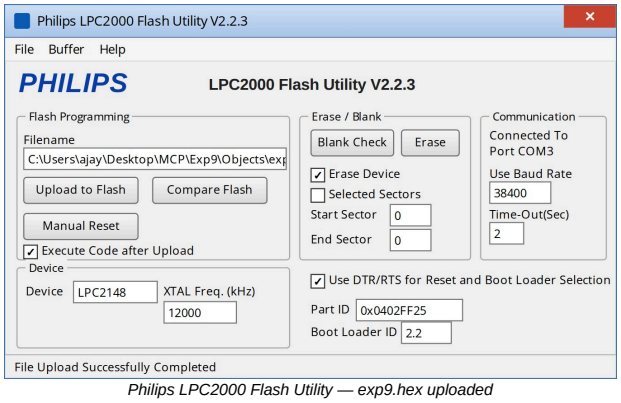
    while(1)
    {
        IO0SET = 0x00010000;
        delay(500000);

        IO0CLR = 0x00010000;
        delay(500000);
    }
}
```

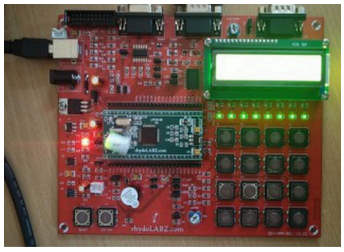
Expected output: The buzzer connected to P0.16 turns ON and OFF repeatedly.

This assumes your buzzer driver is wired to P0.16 and is active-high. Use the board's documented pin assignment; do not connect a buzzer directly to a GPIO if it requires a driver circuit.

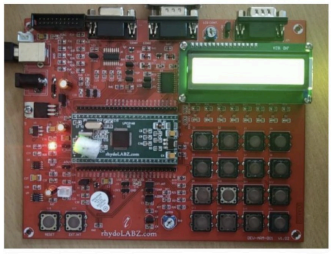
### OUTPUT




*Philips LPC2000 Flash Utility — exp9.hex uploaded*



*Buzzer on P0.16 ON*



*Buzzer OFF*

## Experiment 10 — ARM7 timer-controlled LED blinking

This version follows the timer-based method in your record, using Timer 0 and an LED on P0.7.

Save as `exp10.c`

```c
#include <lpc21xx.h>

void timer0_delay_ms(unsigned int ms)
{
    T0TCR = 0x02;
    T0PR = 14999;
    T0MR0 = ms;
```

```
    T0MCR = 0x04;
    T0TC = 0;
    T0TCR = 0x01;

    while(T0TC < ms);

    T0TCR = 0x00;
}

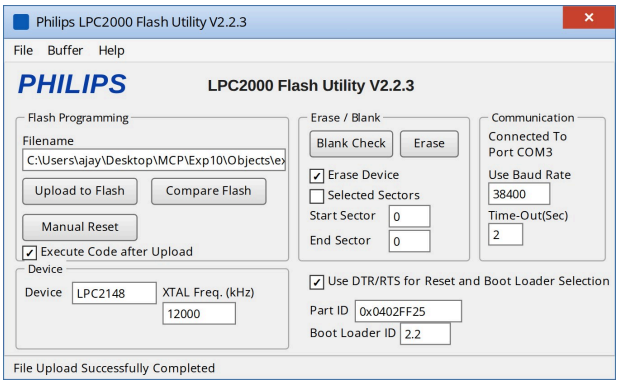
int main(void)
{
    IO0DIR |= 0x00000080;

    while(1)
    {
        IO0SET = 0x00000080;
        timer0_delay_ms(500);

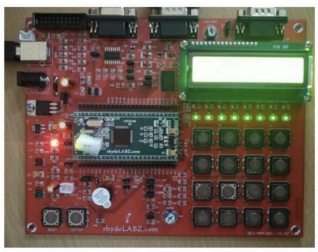
        IO0CLR = 0x00000080;
        timer0_delay_ms(500);
    }
}
```

Expected output: The LED on P0.7 blinks approximately once per second (about 500 ms ON and 500 ms OFF), assuming the timer peripheral clock is 15 MHz.

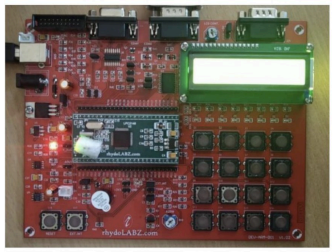
**OUTPUT**




*Philips LPC2000 Flash Utility — exp10.hex uploaded*



*LED on P0.7 ON (500 ms)*



*LED on P0.7 OFF (500 ms)*

**Quick setup reminder**

For all three experiments:

1. Create a separate project for each program.
2. Select the appropriate ARM7/LPC2148 device and toolchain.
3. Build and generate the HEX file.
4. Use the LPC2000 Flash Utility and the appropriate USB/serial programming connection to transfer the HEX file.
5. Reset the board and observe the output.

One important correction: Experiment 8 has two parts in your record—LED interfacing and LCD interfacing. The first code I gave is only the LED part. The LCD program in your record also depends on the supplied `lcd4bit.h` file and the exact LPC2148 board wiring. If that header is missing, the LCD code will not compile until the required functions are included. ....